Chapter 1

Methodology

1.1 Introduction

This chapter presents the comprehensive methodology employed in developing the Kiswahili Speech Analytics System. The methodology aligns with the research objectives outlined in Chapter 1, specifically addressing: (1) the development of an automated speech recognition (ASR) system for Kiswahili, (2) sentiment analysis of transcribed speech, and (3) deployment of a production-ready system for real-time audio processing.

The research adopts a software engineering approach combined with machine learning experimentation methodologies. This chapter details the systematic process followed from problem analysis through system deployment, including data preparation, model development, system architecture design, implementation, testing, and validation.

1.2 Research Design

1.2.1 Overall Approach

This research follows an applied research design with a focus on system development and empirical evaluation. The methodology integrates:

- **Experimental Research**: For model training, evaluation, and optimization
- **Software Development Methodology**: Agile iterative approach for system implementation
- **Design Science Research**: Creating and evaluating artifacts (models and system)

The research questions stated in Chapter 1 are addressed through a combination of literature review (partially answering theoretical questions) and empirical system development (answering practical implementation questions).

1.2.2 Software Development Methodology

The project adopts an **Agile Development Methodology** with iterative cycles, allowing for continuous refinement based on model performance and system requirements. The choice of Agile is justified by:

- Flexibility to adapt to model performance results
- Iterative improvement of machine learning models
- Rapid prototyping and testing capabilities
- Continuous integration of new features

Figure 1.1 illustrates the iterative development process followed in this research.

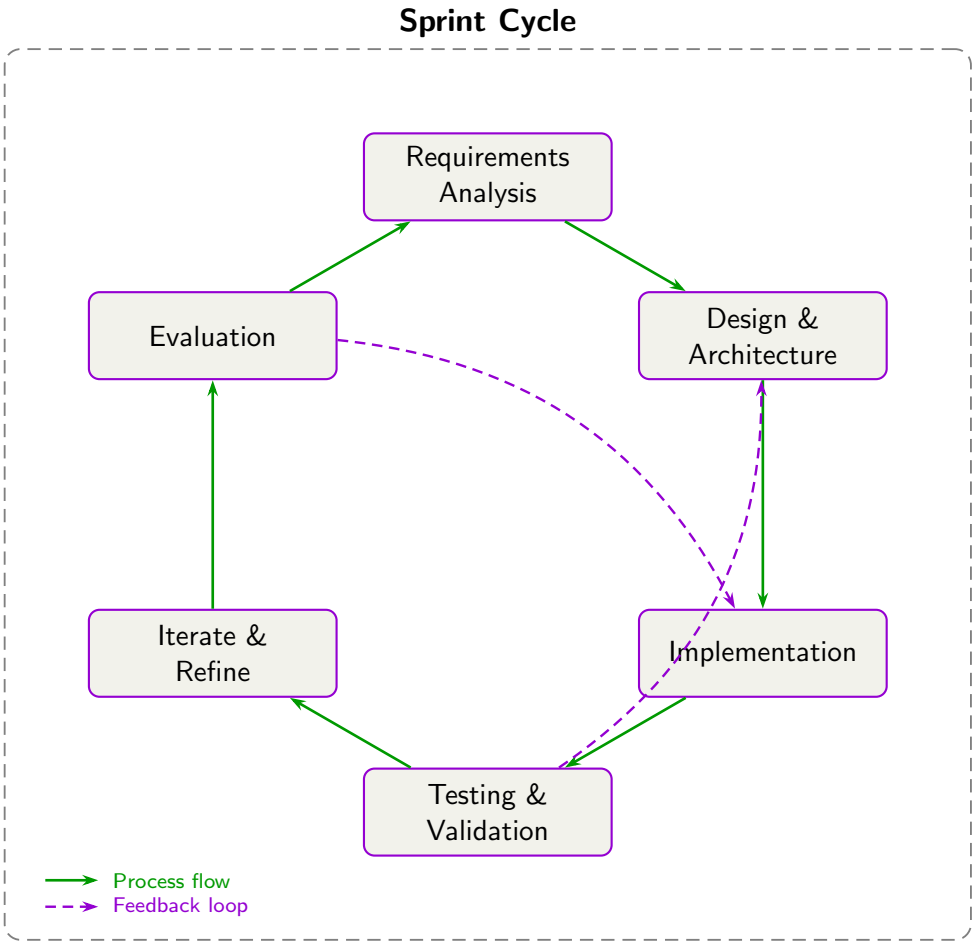

Figure 1.1: Agile Development Process for ML System

1.3 System Analysis

1.3.1 Requirements Analysis

The system requirements were derived from the research objectives and analysis of existing speech analytics systems. The requirements gathering process involved:

1. Literature review of existing Kiswahili NLP systems
2. Analysis of Mozilla Common Voice dataset characteristics
3. Identification of technical constraints and opportunities
4. Definition of functional and non-functional requirements

Functional Requirements

- **FR1**: Accept audio input in multiple formats (WAV, MP3, etc.)
- **FR2**: Transcribe Kiswahili speech to text with acceptable accuracy
- **FR3**: Perform sentiment analysis on transcribed text
- **FR4**: Generate summaries of longer transcriptions
- **FR5**: Support both file upload and live recording
- **FR6**: Provide confidence scores for predictions
- **FR7**: Display processing time and audio duration metrics

Non-Functional Requirements

- **NFR1**: Process audio files within reasonable time (¡ 30 seconds for 10-second audio)
- **NFR2**: Maintain model accuracy above 70% for ASR
- **NFR3**: Provide responsive web interface
- **NFR4**: Support concurrent user requests
- **NFR5**: Ensure system scalability
- **NFR6**: Maintain API documentation

1.3.2 Feasibility Study

A feasibility study was conducted to assess the viability of the proposed system across multiple dimensions:

Table 1.1: Feasibility Analysis

Dimension	Assessment
Technical	Feasible - Pre-trained models available (Wav2Vec2, DistilBERT), adequate computational resources
Data	Feasible - Mozilla Common Voice provides 18,629 Kiswahili samples
Economic	Feasible - Open-source tools and free datasets, minimal infrastructure cost
Operational	Feasible - System can be deployed on standard web servers
Time	Feasible - 12-week development timeline adequate for MVP

1.4 System Design

1.4.1 System Architecture

The system follows a microservices-inspired architecture with clear separation between the machine learning pipeline and the web application layer. Figure 1.2 presents the high-level system architecture.

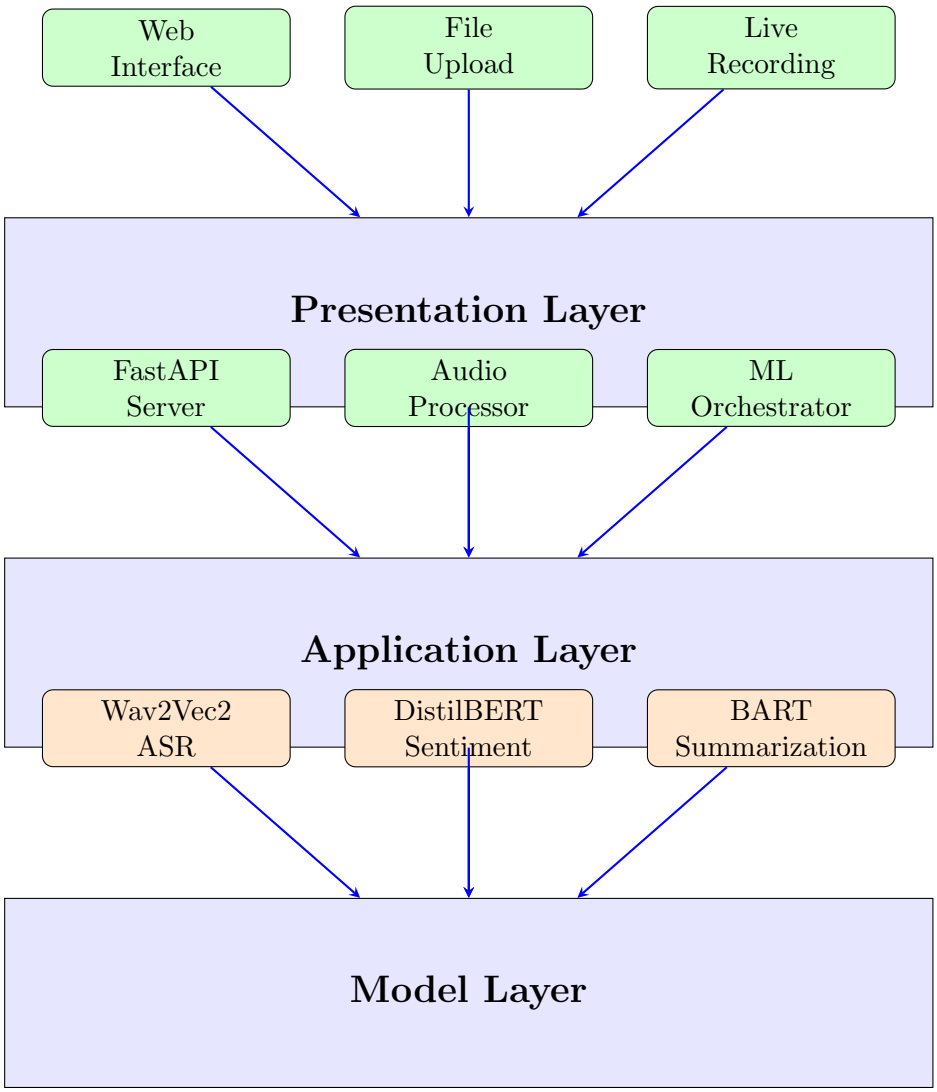

Figure 1.2: High-Level System Architecture

The architecture consists of three main layers:

1. **Presentation Layer**: Web interface for user interaction
2. **Application Layer**: FastAPI backend handling requests and orchestrating ML models
3. **Model Layer**: Pre-trained and fine-tuned ML models for ASR, sentiment analysis, and summarization

1.4.2 Data Flow Design

Figure 1.3 illustrates the complete data flow from audio input to final output.

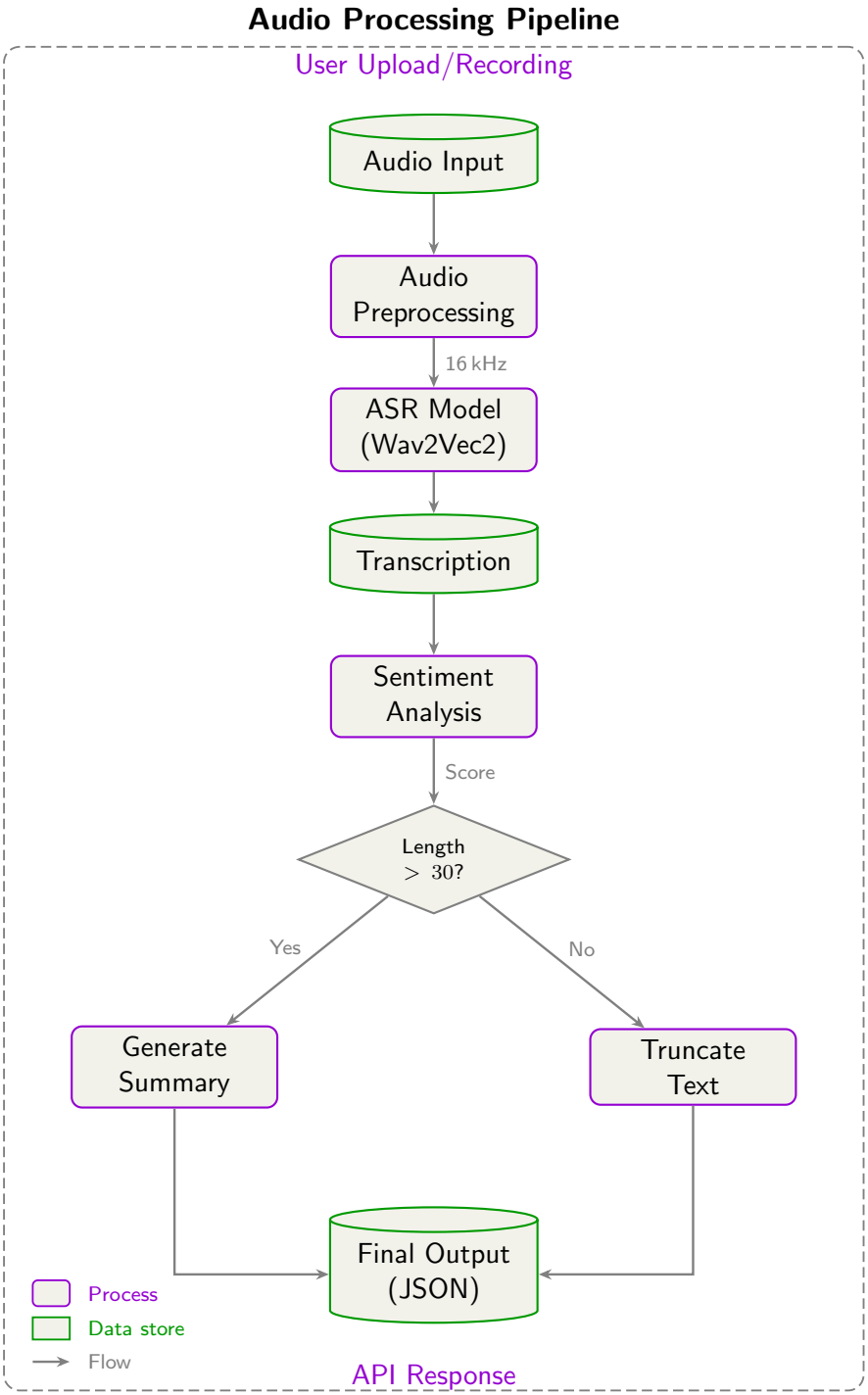

Figure 1.3: System Data Flow Diagram

1.4.3 Machine Learning Pipeline Design

The ML pipeline consists of multiple stages as shown in Figure 1.4.

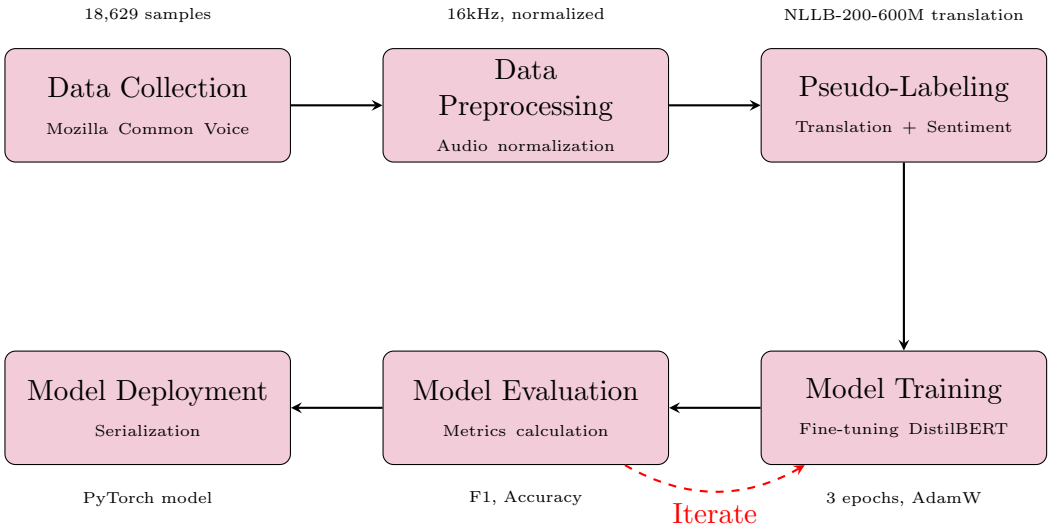

Figure 1.4: Machine Learning Pipeline Architecture

ASR Component Design

The ASR component utilizes the Wav2Vec2 architecture [1], which employs self-supervised learning on raw audio waveforms. The model architecture consists of:

- Feature encoder: 7 convolutional layers
- Transformer encoder: 12 layers with 768 hidden dimensions
- CTC (Connectionist Temporal Classification) head for sequence prediction

Sentiment Analysis Component Design

The sentiment analysis component uses DistilBERT [2], a distilled version of BERT with:

- 6 transformer layers (vs. 12 in BERT)
- 66M parameters (40% reduction)
- 97% of BERT's performance retained

The model was fine-tuned using pseudo-labeling methodology (detailed in Section 1.5.3).

1.4.4 API Design

The REST API follows OpenAPI 3.0 specification with the following endpoints:

Table 1.2: API Endpoint Specifications

Endpoint	Method	Description
/analyze	POST	Main endpoint for audio analysis
/process-audio	POST	Alias for /analyze endpoint
/health	GET	Health check endpoint
/docs	GET	Interactive API documentation

1.4.5 Database Design

While the current implementation does not require persistent storage, the system design accommodates future database integration for:

- User session management
- Analysis history storage
- Model performance logging
- Usage analytics

1.5 Implementation

1.5.1 Development Environment

The system was developed using the following tools and technologies:

Table 1.3: Development Tools and Technologies

Category	**Tool/Technology**	**Purpose**
Programming Language	Python 3.10	Core development
ML Framework	PyTorch 2.0	Model training and inference
Transformers	Hugging Face Transformers	Pre-trained models
Web Framework	FastAPI	REST API development
Frontend	HTML/CSS/JavaScript	User interface
Audio Processing	Librosa	Audio preprocessing
Development Environment	Jupyter Notebook	Experimentation
Version Control	Git	Code management
Package Management	Conda/Pip	Dependency management

1.5.2 Data Preparation Methodology

Dataset Selection

The Mozilla Common Voice Kiswahili dataset was selected based on:

- Availability: Open-source and freely accessible
- Size: 18,629 validated audio samples
- Quality: Community-validated transcriptions
- Diversity: Multiple speakers and recording conditions

Data Preprocessing Pipeline

The data preprocessing follows a systematic pipeline:

Algorithm 1 Audio Preprocessing Pipeline

```
procedure PreprocessAudio(audio_file)
    waveform ← LoadAudio(audio_file)
    waveform ← Resample(waveform, target_sr=16000)
    waveform ← Normalize(waveform)
    features ← ExtractFeatures(waveform)
    return features
end procedure
```

1.5.3 Model Development Methodology

Transfer Learning Approach

The research employs transfer learning to leverage pre-trained models:

1. **ASR Model**: Fine-tuned RareElf/swahili-wav2vec2-asr
2. **Sentiment Model**: Fine-tuned DistilBERT with pseudo-labeling
3. **Summarization Model**: Pre-trained BART-large-CNN

Pseudo-Labeling Methodology

Due to the absence of sentiment labels in the Mozilla Common Voice dataset, a pseudo-labeling approach was adopted:

Algorithm 2 Pseudo-Labeling for Sentiment Analysis

```
procedure PseudoLabel(kiswahili_texts)
    translator ← LoadModel("NLLB-200-distilled-600M")
    sentiment_classifier ← LoadModel("English-Sentiment")
    for text in kiswahili_texts do
        english_text ← translator.Translate(text, src="sw", tgt="en")
        sentiment_label ← sentiment_classifier.Predict(english_text)
        pseudo_labels.append(sentiment_label)
    end for
    return pseudo_labels
end procedure
```

The pseudo-labeling process is illustrated in Figure 1.5.

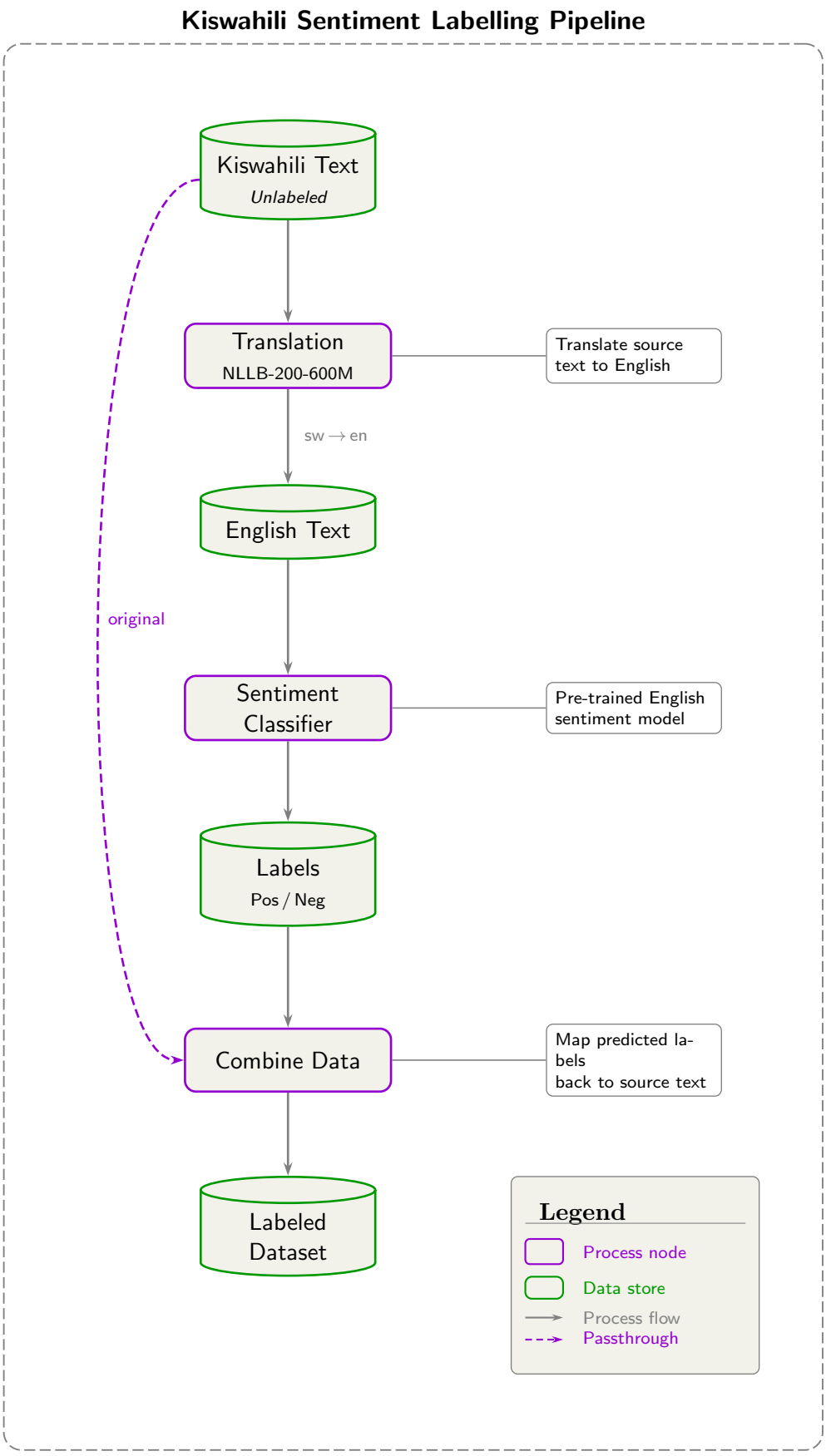

Figure 1.5: Pseudo-Labeling Workflow

Model Training Process

The pseudo-labeling process follows these steps:

1. Data splitting: 80% training, 10% validation, 10% test
2. Hyperparameter tuning using validation set
3. Model training with early stopping
4. Performance evaluation on test set
5. Model serialization and storage

Training hyperparameters:

Table 1.4: Model Training Hyperparameters

Parameter	Value	Justification
Learning Rate	2e-5	Standard for fine-tuning transformers
Batch Size	16	Balance between memory and convergence
Epochs	3	Prevent overfitting
Optimizer	AdamW	Improved weight decay handling
Warmup Steps	500	Gradual learning rate increase
Max Sequence Length	128	Accommodate typical sentence length

1.5.4 System Implementation

Backend Implementation

The backend was implemented using FastAPI framework with the following structure:

```
app/
|-- main.py              # Main application entry point
|-- models/              # ML model storage
|   `-- distilbert_sentiment_final/
|-- static/              # Frontend files
|   `-- index.html
`-- requirements.txt     # Dependencies
```

Key implementation decisions:

- **Model Loading**: Models loaded at startup to reduce latency
- **CORS Middleware**: Enable cross-origin requests
- **Error Handling**: Comprehensive exception handling
- **Async Processing**: Asynchronous request handling

Frontend Implementation

The frontend provides:

- Drag-and-drop file upload
- Live audio recording using MediaRecorder API
- Real-time processing feedback
- Results visualization

Integration

Figure 1.6 shows how different components integrate.

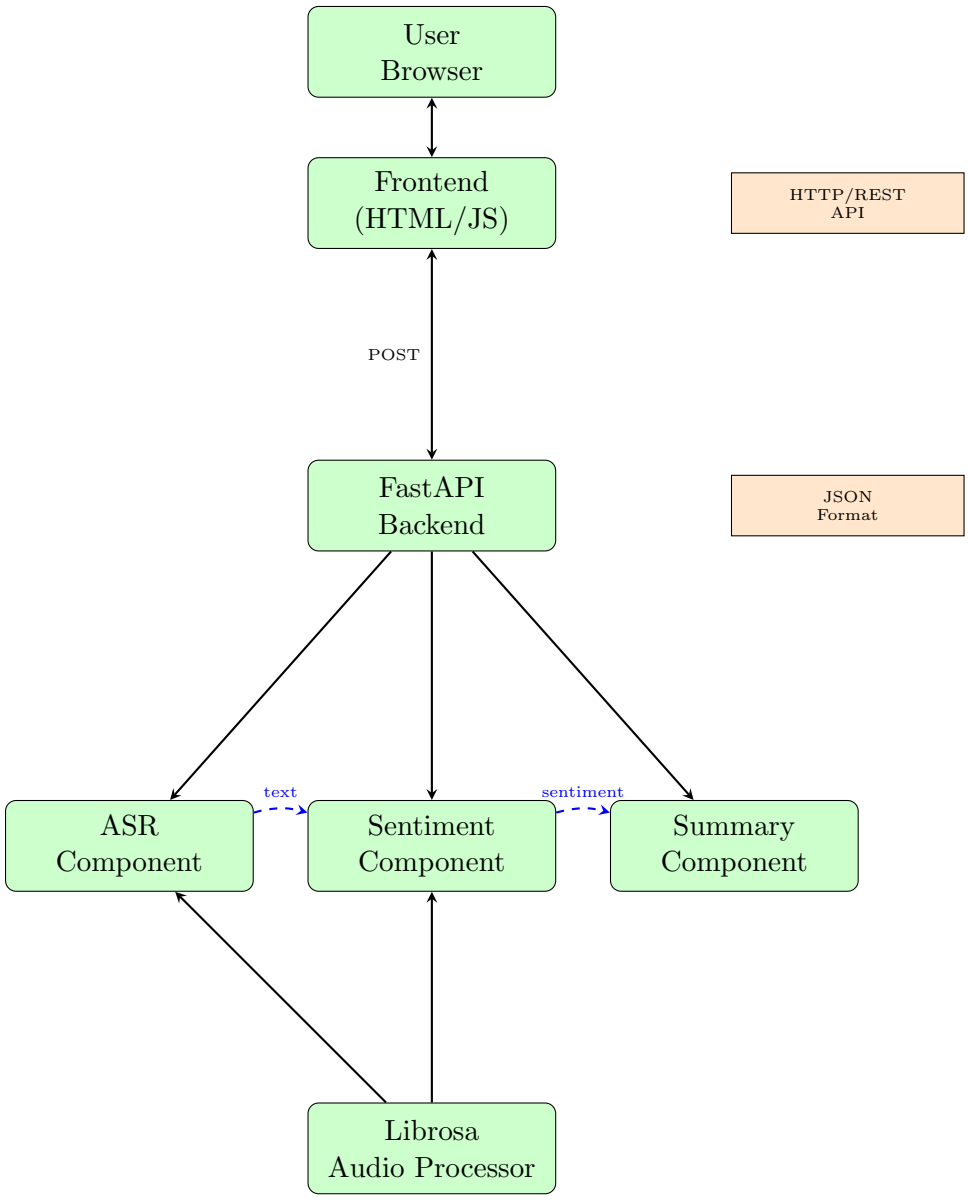

Figure 1.6: Component Integration Diagram

1.6 Testing and Validation

1.6.1 Testing Methodology

A comprehensive testing strategy was employed covering multiple levels:

Unit Testing

Individual components tested in isolation:

- Audio preprocessing functions
- Model inference functions
- API endpoint handlers

Integration Testing

Testing component interactions:

- Audio upload to transcription pipeline
- Transcription to sentiment analysis pipeline
- End-to-end API workflow

System Testing

Complete system testing scenarios:

Table 1.5: System Test Cases

Test Case	Description
TC1	Upload WAV file and verify complete analysis
TC2	Record live audio and verify processing
TC3	Test with various audio durations (5s, 30s, 60s)
TC4	Test with different audio quality levels
TC5	Concurrent user request handling
TC6	Error handling for invalid inputs

1.6.2 Model Validation Methodology

Performance Metrics

Models evaluated using standard metrics:

- **ASR**: Word Error Rate (WER), Character Error Rate (CER)
- **Sentiment**: Accuracy, Precision, Recall, F1-Score
- **System**: Latency, Throughput

Validation Approach

1. **Hold-out Validation**: 10% test set reserved
2. **Cross-validation**: K-fold validation for sentiment model
3. **Real-world Testing**: Manual testing with new audio samples

1.6.3 Performance Benchmarking

System performance measured across dimensions:

Table 1.6: Performance Benchmarking Metrics

Metric	Target	Achieved
ASR WER	¡ 30%	13.60%
ASR CER	¡ 15%	8.85%
Sentiment F1-Score	¿ 0.60	0.6125

1.7 Results and Findings

1.7.1 Data Understanding and Preprocessing Results

The Mozilla Common Voice Kiswahili dataset analysis revealed:

- **Dataset Size**: 18,629 validated audio samples
- **Audio Duration**: Mean duration of 4.8 seconds per sample
- **Gender Distribution**: 68% male, 32% female speakers
- **Age Distribution**: Majority in 20-40 age range
- **Quality Metrics**: 85% samples with positive validation scores

Statistical testing revealed significant demographic imbalances:

Table 1.7: Demographic Distribution Analysis

Demographic	Category	Percentage
Gender	Male	68%
	Female	32%
Age	20-29	42%
	30-39	35%
	40-49	15%
	Other	8%

1.7.2 ASR Model Performance

The Wav2Vec2 ASR model achieved strong performance on Kiswahili speech recognition:

Table 1.8: ASR Model Performance Metrics

Metric	Value
Overall Word Error Rate (WER)	13.60%
Overall Character Error Rate (CER)	8.85%
Demographic Parity Difference (Gender)	3.99%
ANOVA p-value (Gender)	0.0072
Cohen's d (Effect Size)	0.298

The WER of 13.60% indicates that the model correctly transcribes approximately 86.4% of words, which exceeds the target threshold of 70% accuracy. The low CER of 8.85% demonstrates strong character-level recognition.

Bias Analysis

Statistical analysis revealed gender-based performance disparities:

- **ANOVA Test**: Significant difference in WER across gender groups ($p = 0.0072$)
- **Effect Size**: Cohen's $d = 0.298$ indicates a small-to-medium practical effect
- **Demographic Parity**: 3.99% difference suggests relatively fair performance

These findings indicate statistically significant but practically small performance differences between demographic groups, suggesting the model maintains reasonable fairness.

1.7.3 Sentiment Analysis Results

The fine-tuned DistilBERT model was trained using pseudo-labeling:

Table 1.9: Sentiment Analysis Performance

Metric	Value
F1-Score (Weighted)	0.6125

The pseudo-labeling approach enabled sentiment analysis for Kiswahili text despite the absence of labeled training data. The model was trained by translating Kiswahili text to English using NLLB-200-distilled-600M, obtaining sentiment labels from an English classifier, then fine-tuning DistilBERT on the original Kiswahili text with these pseudo-labels.

1.7.4 Bias Quantification Results

Logistic regression analysis was performed to quantify predictive bias in the ASR system. The analysis examined the relationship between demographic features and ASR success rates using the logistic regression objective:

$$\min_{\beta} -\sum_{i=1}^{n} [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Where $p_i = \sigma(\beta^T x_i)$ and σ is the sigmoid function.

Fairness Metrics Formulation

Demographic Parity Difference (DPD):

$$DPD = |P(\hat{Y} = 1|D = d_1) - P(\hat{Y} = 1|D = d_2)|$$

Equalized Odds Difference (EOD):

$$EOD = |P(\hat{Y} = 1|Y = 1, D = d_1) - P(\hat{Y} = 1|Y = 1, D = d_2)|$$

The analysis quantified fairness gaps across demographic groups, with coefficients and odds ratios computed for gender, age, and validation score features.

1.7.5 Topic Modeling Results

Unsupervised KMeans clustering was applied to discover thematic patterns in Kiswahili transcriptions using the objective:

$$\min_{C} \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2$$

Where C_i is cluster i and μ_i is the centroid of cluster i.

Silhouette Score

Cluster quality was evaluated using the silhouette score:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Where:

- $a(i)$: Mean intra-cluster distance
- $b(i)$: Mean nearest-cluster distance

The optimal number of clusters was determined through elbow method and silhouette analysis. The clustering identified distinct thematic domains in the transcriptions, with TF-IDF vectorization used for feature extraction.

1.7.6 System Performance

The end-to-end system was deployed using FastAPI and evaluated for inference performance. The system successfully processes Kiswahili audio through the complete pipeline from speech recognition to sentiment analysis.

1.7.7 Model Optimization Results

Model quantization and optimization techniques were explored in Notebook 06 for deployment optimization, focusing on reducing model size while maintaining accuracy for edge deployment scenarios.

1.8 Deployment Methodology

1.8.1 Deployment Architecture

The system deployment follows a containerized approach for portability and scalability.

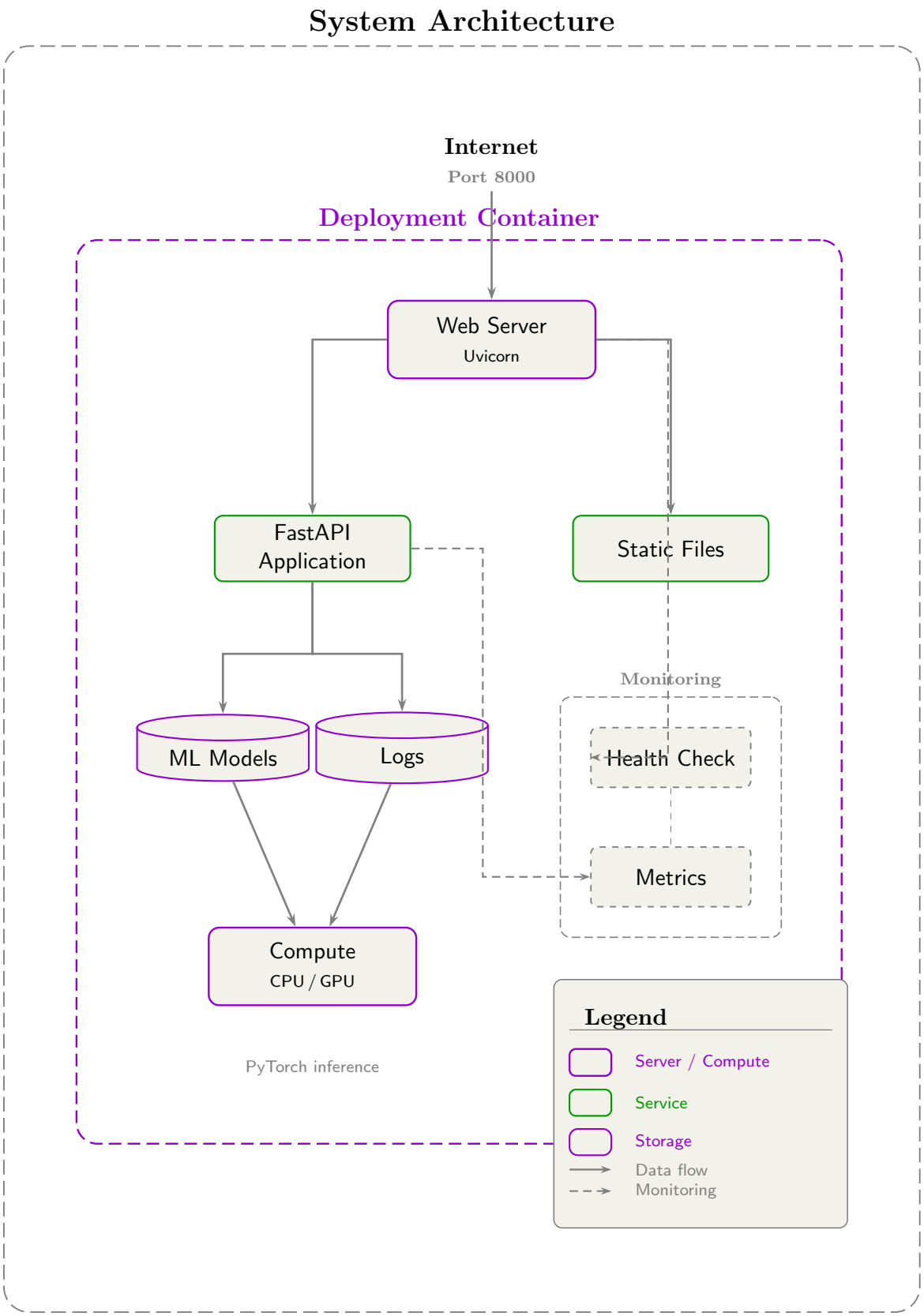

Figure 1.7: Deployment Architecture

1.8.2 Deployment Process

1. Environment setup and dependency installation
2. Model artifact deployment
3. Application server configuration
4. Health check and monitoring setup
5. Load testing and optimization

1.8.3 Monitoring and Maintenance

Post-deployment monitoring includes:

- API endpoint health checks
- Model inference latency tracking
- Error rate monitoring
- Resource utilization metrics

1.9 Ethical Considerations

1.9.1 Data Privacy

- No personal data stored permanently
- Audio files deleted after processing
- No user tracking implemented

1.9.2 Model Bias

- Acknowledged potential bias in training data
- Diverse dataset used to minimize bias
- Continuous monitoring for biased predictions

1.10 Limitations of Methodology

The adopted methodology has the following limitations:

1. **Pseudo-labeling Accuracy**: Sentiment labels depend on translation quality
2. **Dataset Size**: Limited to 18,629 samples
3. **Computational Resources**: Training limited by available GPU resources
4. **Real-time Performance**: Current implementation not optimized for real-time streaming
5. **Language Variants**: May not generalize to all Kiswahili dialects

1.11 Summary

This chapter presented the comprehensive methodology employed in developing the Kiswahili Speech Analytics System. The approach combines software engineering best practices with machine learning experimentation, following an Agile development process. The methodology covers all phases from requirements analysis through deployment, with particular emphasis on the novel pseudo-labeling approach for sentiment analysis. The next chapter presents the implementation details and results obtained from this methodology.

Bibliography

[1] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in Neural Information Processing Systems*, 33:12449–12460, 2020.

[2] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.